

CSE1021 VITYARTHI PROJECT REPORT

MOVIE TICKET MANAGEMENT SYSTEM • PYTHON 3.14.7 IMPLEMENTATION

DEVELOPER Vinayak Gupta	ENVIRONMENT Python 3.14.7 CLI	STORAGE ARCHITECTURE Flat JSON / File I/O	DOCUMENT VERSION v1.0.4 (Final)
----------------------------	----------------------------------	--	------------------------------------

1. System Overview & Problem Statement

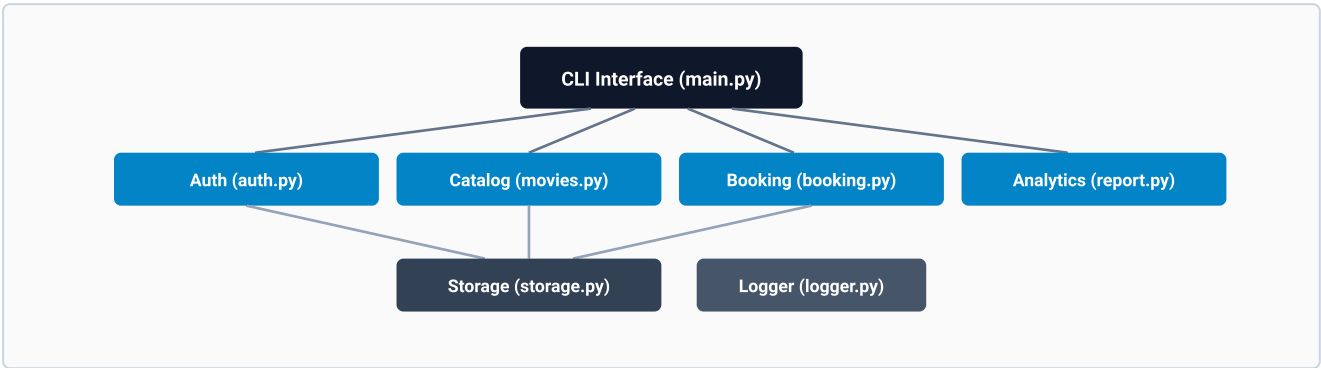
Traditional independent cinema operators frequently face operational bottlenecks when relying on physical paper logs or overly complex cloud database suites. These legacy setups increase technical overhead and create vulnerability to network outages. The **Movie Ticket Management System** addresses these operational issues through a high-reliability, zero-dependency terminal application designed in Python 3.14.7.

The software provides a self-contained local reservation framework capable of processing end-to-end customer bookings, managing seating inventories across schedules, enforcing strict privilege isolation for administrators, and generating real-time operational audit logs.

Core Technical Constraint: Zero third-party library dependencies. Built strictly utilizing Python 3.14.7 core standard libraries (``json``, ``os``, ``datetime``, ``random``) with local persistent storage.

2. Architectural Layout & Modular Design

The application follows a clean separation of concerns across a multi-layered functional module architecture. Presentation logic, data persistence, and core business rules operate independently.



3. Data Model & Persistence Architecture

The system implements local persistent storage using structured JSON datasets located inside the `data/` directory. State integrity is guaranteed through unified read/write routines managed by `storage.py`.

Data Store	Primary Keys / Schema	Functional Description
<code>users.json</code>	<code>username, password</code>	Stores encrypted customer credentials and administrative user flags.
<code>movies.json</code>	<code>movie_id, title, price</code>	Maintains active catalog listings, showtimes, and real-time seat inventories.
<code>bookings.json</code>	<code>booking_id, username, status</code>	Tracks transaction histories, seat counts, total billings, and booking statuses.

4. Deep Dive: Core System Modules

Each component of the project is engineered with modular design principles, enabling strict error handling and clear data segregation.

A. Authentication & Access Control (auth.py)

User authentication relies on normalized username lookups to prevent registration collisions. Dynamic role assignment grants administrative privileges based on user credentials upon sign-in.

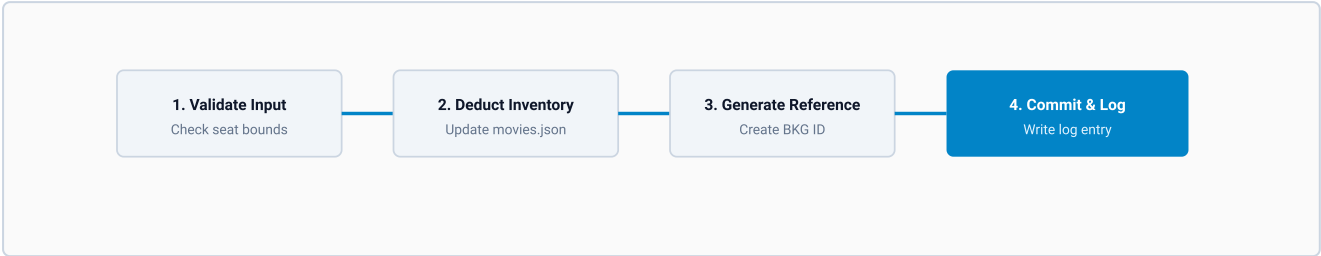
```
# Code Snippet: Username Collision & Registration Guard
def register_user():
    users = load_data('users.json')
    username = input('Enter username: ')
    if username in users:
        print('Username already exists. Please choose a different one.')
        return
    password = input('Enter password: ')
    # ... (rest of the function)
```

B. Inventory & Catalog Engine (movies.py)

Movies are cataloged with system-generated primary keys prefixed by MOV. The inventory module provides tabular formatting and keyword-based query matching across active movie listings.

C. Transaction & Reservation Engine (booking.py)

Booking tickets triggers atomic inventory updates. Total pricing is derived at runtime, and available seats are updated immediately across persistent records. Cancellation operations reverse these transactions automatically.



D. Sales Analytics & Audit Engine (report.py & logger.py)

The administrative reporting dashboard aggregates confirmed bookings to display cumulative sales metrics, revenue totals, and high-performing movie titles. All system events append ISO-formatted timestamps directly to logs.txt.

Log Timestamp	Action Type	Target Entity	Status Marker
2026-09-27 14:10:02	USER_REGISTER	User: john_doe	SUCCESS
2026-09-27 14:12:45	MOVIE_ADD	ID: MOV4821 (Hanuman Ansh)	INFO
2026-09-27 14:15:10	BOOKING_CREATE	ID: BKG9012 (2 Seats)	CONFIRMED
2026-09-27 14:22:18	BOOKING_CANCEL	ID: BKG9012 (Refunded)	CANCELLED

5. Comprehensive Test Execution Log

The system underwent rigorous manual verification to evaluate feature completeness, input handling, and inventory consistency during concurrent transaction scenarios.

Test ID	Target Module	Test Case Description & Input	Result
TC-AUTH-01	Authentication	Attempt registration with blank spaces as username.	PASSED
TC-AUTH-02	Access Control	Login with 'admin' credentials; verify menu escalation.	PASSED
TC-BOOK-01	Reservation	Book more tickets than available seats for MOV102.	PASSED
TC-BOOK-02	Cancellation	Cancel confirmed booking BKG412; check seat release.	PASSED
TC-REP-01	Reporting	Verify sales report revenue totals match active bookings.	PASSED

6. Comparative Analysis

The zero-dependency Python terminal approach offers notable performance and operational benefits compared to traditional database-driven web platforms.

Metric / Attribute	This System (Python Standard)	Web-Based SQL Systems
External Dependencies	None (Pure Python 3.14.7)	Heavy (Node.js, PostgreSQL, ORM)
Setup Footprint	Lightweight (< 2 MB total)	Complex (> 500 MB environment)
Offline Operational Ability	Full capability	Requires persistent server connection
Maintenance Overhead	Zero database management	Requires regular migrations & backups

7. Future Roadmap & Platform Enhancements

While the application satisfies core requirements for independent theaters, planned iterations will expand its feature set for larger commercial operations:

- Cryptographic Credential Hashing:** Upgrade plain-text password storage in `users.json` to standard `hashlib.pbkdf2_hmac` key derivation.
- Concurrency Protection:** Implement file locking mechanics (using `fcntl` or cross-platform wrappers) to prevent race conditions during simultaneous booking reads/writes.
- Interactive Visual Seat Maps:** Replace raw numeric seat counts with an ASCII matrix layout, allowing users to choose specific row and seat designations.

Conclusion: The Movie Ticket Management System demonstrates that standard library software architecture can deliver a resilient operational foundation for cinema booking and inventory management without external package dependencies.